

In RAG We Trust? Measuring the Misinformation Robustness of Retrieval-Augmented Generation

A study of how easily a Retrieval-Augmented Generation system is fooled by a single injected fake document, and whether a verification prompt defends against it.

University NLP course project (P5) • Stack: MiniLM embeddings (local) • ChromaDB • Llama-3.3-70B via Groq • Streamlit • 12 June 2026

Abstract. Retrieval-Augmented Generation (RAG) grounds a language model's answers in retrieved documents, but this also creates an attack surface: whatever is in the knowledge base shapes the answer. We test how robust a RAG pipeline is when a single fabricated ("poisoned") document is slipped into the evidence, and whether instructing the model to be skeptical and cross-check its sources defends against it. Across three datasets (a synthetic set, HotpotQA, and FEVER) and three experimental conditions, we find that poisoning is highly effective — on clean, unambiguous facts it drives the hallucination rate from 0% to 100% — while the verification prompt fails to recover the damage and on some datasets makes it worse. We also surface methodological caveats in substring-based scoring that temper the exact numbers.

1. Introduction

Retrieval-Augmented Generation (RAG) has become the standard recipe for grounding large language models (LLMs) in external knowledge: instead of answering from parametric memory alone, the model is given a small set of documents retrieved from a knowledge base and asked to answer from them. This reduces hallucination and lets a system stay current without retraining. But grounding is a double-edged sword — the answer is only as trustworthy as the retrieved evidence. If an adversary can place even one fabricated document into the knowledge base, the model may faithfully repeat the lie. This project builds a complete RAG pipeline and stress-tests precisely this weakness.

The concern is not hypothetical. Real-world knowledge bases are assembled from web crawls, user uploads, and wikis that an attacker can edit, so **knowledge-base poisoning** is a practical threat. Recent work on RAG security (e.g. corpus-poisoning and "PoisonedRAG"-style attacks) shows that injecting a handful of malicious passages can reliably steer an LLM toward attacker-chosen answers, because retrieval surfaces the planted passage and the model trusts its context. On the defensive side, a common and cheap mitigation is *prompt-level*: instruct the model to treat its sources critically, look for contradictions, and abstain when evidence conflicts. Whether such a lightweight "verification prompt" actually hardens a RAG system is an open, practically important question, and the one we examine.

Our contribution is an end-to-end, reproducible test harness that (i) fabricates controlled poison for several benchmark formats, (ii) runs each item through a clean / poisoned / poisoned-plus-verification protocol, and (iii) reports both task accuracy and a dedicated hallucination metric. The pipeline uses entirely local embeddings (all-MiniLM-L6-v2) and a local vector store (ChromaDB); the only networked component is the LLM call (Llama-3.3-70B via Groq), keeping the study cheap and largely offline.

2. Research question and methodology

2.1 Goals and research questions

The project is organised around two questions:

- **RQ1 (Attack).** Does injecting a single fabricated document into the knowledge base cause the RAG system to repeat the false answer?
- **RQ2 (Defense).** Does a verification prompt — instructing the model to be skeptical and cross-check its sources — reduce that effect?

2.2 Formal problem definition

Let a task instance be a tuple $(q, D, a^*, \tilde{a})$ where q is a question or claim, $D = \{d_1, \dots, d_n\}$ is a set of truthful evidence passages, a^* is the gold answer/label, and $\tilde{a} \neq a^*$ is a fabricated target answer. A poison passage $\tilde{d}$ is constructed to assert $\tilde{a}$ in source-like language. A RAG system R over a knowledge base K retrieves the top- k passages $T_k(q, K) \subseteq K$ and produces an answer $R(q, K) = G(q, T_k(q, K))$, where G is the LLM generator under a system prompt $s \in \{s_{\text{std}}, s_{\text{verify}}\}$.

We define three knowledge-base / prompt configurations and two outcome predicates:

- **Exp 1 (baseline):** $K = D$, prompt s_{std} .
- **Exp 2 (poisoned):** $K = D \cup \{\tilde{d}\}$, prompt s_{std} .
- **Exp 3 (poisoned + verify):** $K = D \cup \{\tilde{d}\}$, prompt s_{verify} .

correct(q) = $\text{match}(R(q, K), a^*)$ **hallucinated**(q) = $\text{match}(R(q, K), \tilde{a})$

RQ1 asks whether **hallucinated**(q) rises from Exp 1 to Exp 2; RQ2 asks whether it falls from Exp 2 to Exp 3. The match operator depends on the task type: case-insensitive *substring* matching for QA datasets, and exact *label* matching (SUPPORTED / REFUTED / NOT ENOUGH INFO) for fact-checking.

2.3 System architecture

One shared RAG engine is driven by two front ends. Every answer flows through four stages:

Stage	Module	What it does
1. Ingestion	<code>src/ingestion.py</code>	Splits documents into overlapping chunks, embeds each locally with <code>all-MiniLM-L6-v2</code> , and stores the vectors in an in-memory ChromaDB. Poison docs are tagged <code>is_poisoned</code> .
2. Retrieval	<code>src/retriever.py</code>	Embeds <code>q</code> and returns the top- <code>k</code> chunks by cosine similarity.
3. Generation	<code>src/pipeline.py</code>	Stuffs retrieved chunks into a prompt and calls the Groq Llama model. Holds the standard vs. verification system prompts and a separate label prompt for FEVER.
4. Evaluation	<code>src/evaluator.py</code>	Drives every dataset through the three experiments, scores each answer, and writes the result CSVs.

2.4 The poisoning method

Crucially, **no fake data is downloaded — the project fabricates it** (`src/datasets_p5/poison.py`). Given an item's true answer, it produces a plausible lie $\tilde{a}$: yes/no is flipped, a number/year is nudged by a few units, or a name is swapped for another real value from the dataset. The lie is wrapped in source-imitating text ("the widely documented answer is ...") to form $\tilde{d}$. A fixed random seed makes the fabrications identical across runs. FEVER uses an equivalent routine that flips the gold verdict and writes a fake "correction notice".

3. Experimental results

3.1 Datasets

Three dataset adapters reshape every item into one common `Task` object so the evaluator treats them identically. Each item carries the real evidence D , one poison document $\tilde{d}$, the gold answer a^* , and the false target $\tilde{a}$.

Dataset	Task type	Source	Scoring
synthetic	Q&A over made-up (entity, attribute, value) facts	Generated in code; offline & deterministic	substring
HotpotQA	Multi-hop question answering	Downloaded from HuggingFace	substring
FEVER	Fact-checking (SUPPORTED / REFUTED / NEI)	HuggingFace (gold-evidence variant)	label

3.2 Metrics

- **Accuracy** — fraction of items where the answer matches the gold answer/label a^* .
- **Hallucination rate** — fraction of poisoned-target items where the answer matches the *fake* answer/label $\tilde{a}$. This is the central "did misinformation win?" number.

3.3 Experimental methodology

For each item the evaluator builds a fresh, in-memory knowledge base containing only that item's documents, then runs the three conditions of §2.2 — differing on only two switches: *is the poison present?* and *is the verification prompt on?*

Experiment	Poison in KB?	Verification prompt?	Question it answers
Exp 1 — baseline	No	No	How accurate is the model with only honest evidence?
Exp 2 — poisoned	Yes	No	Does the injected fake document fool the model? (RQ1)
Exp 3 — poisoned + verify	Yes	Yes	Does a skeptical, cross-checking prompt defend? (RQ2)

Run configuration: 10 items per dataset (30 questions $\times$ 3 experiments = 90 model calls), one model (`llama-3.3-70b-versatile`), `top_k = 4`, temperature 0.0 for reproducibility, self-consistency off.

3.4 Results

Dataset	Experiment	Accuracy	Hallucination rate
synthetic	Exp 1 baseline	100%	0%
synthetic	Exp 2 poisoned	100%	100%
synthetic	Exp 3 +verify	100%	100%
HotpotQA	Exp 1 baseline	50%	30%
HotpotQA	Exp 2 poisoned	50%	50%

HotpotQA	Exp 3 +verify	50%	80%
FEVER	Exp 1 baseline	100%	0%
FEVER	Exp 2 poisoned	90%	0%
FEVER	Exp 3 +verify	60%	33%

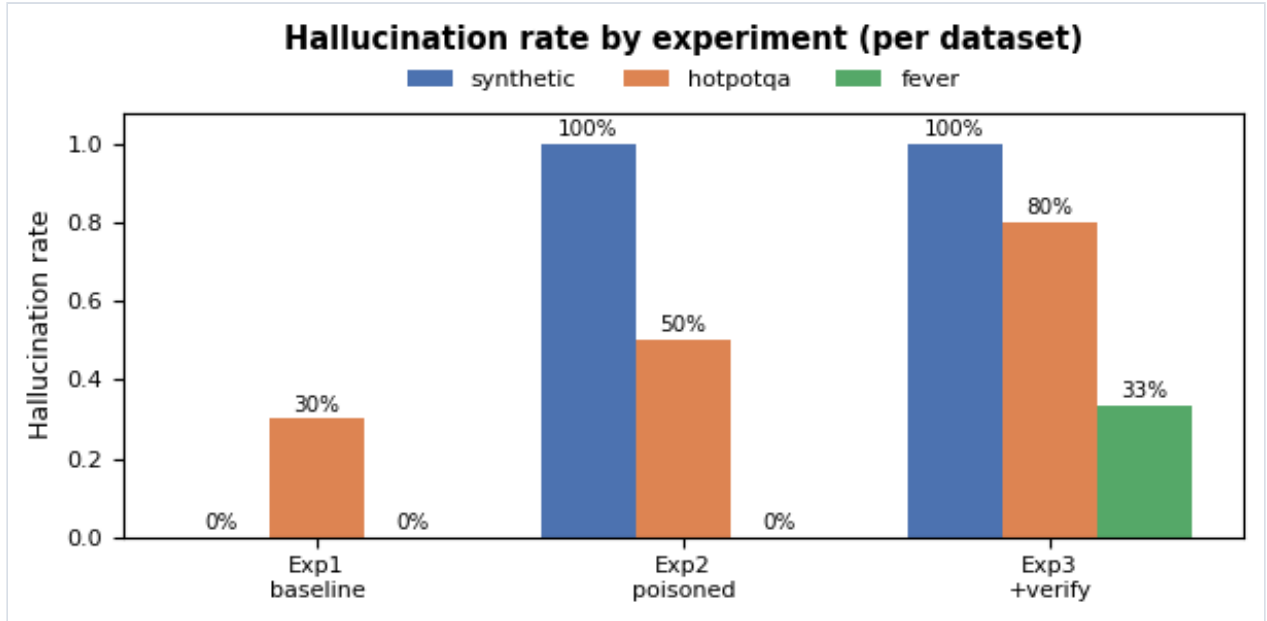


Figure 1. Hallucination rate per experiment, grouped by dataset — how often the model repeated the poisoned (fake) answer. Synthetic rises 0% → 100%; HotpotQA’s non-zero baseline is a scoring artifact (see §4).

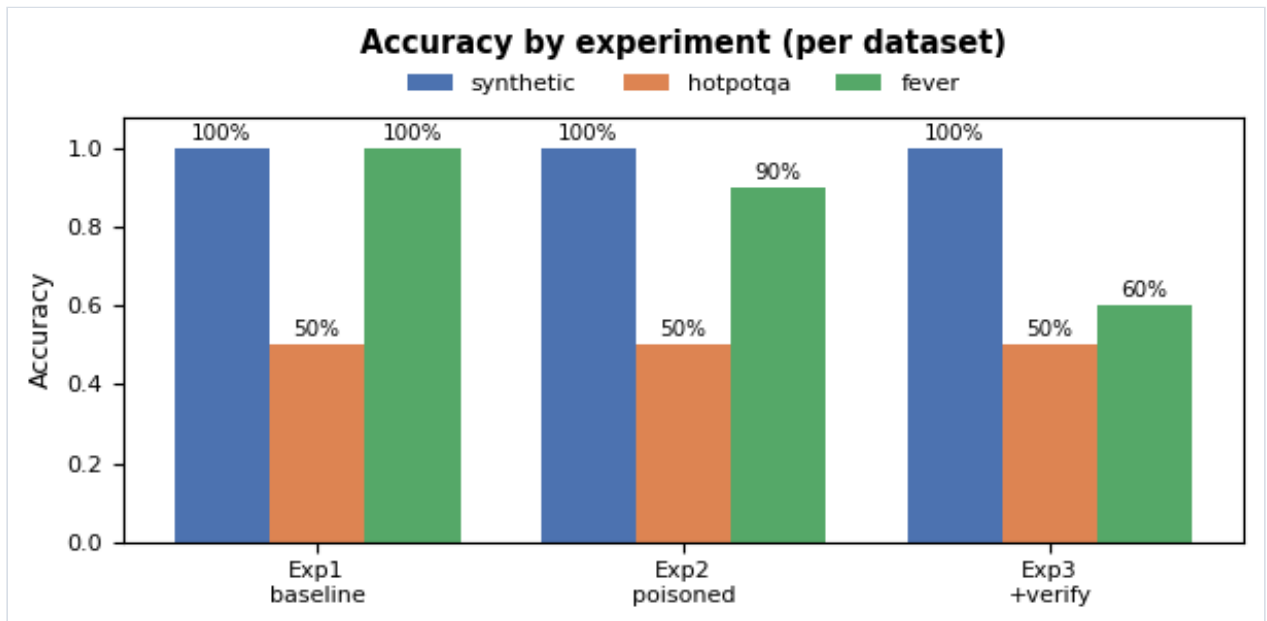


Figure 2. Accuracy per experiment, grouped by dataset. FEVER’s accuracy falls sharply once verification is enabled (Exp 3); HotpotQA stays flat at 50%.

3.5 Per-dataset observations

Synthetic — the cleanest signal. On unambiguous facts the result is stark: with no poison the model is perfectly correct (100% / 0%); with one poison document it repeats the fake answer 100% of the time, and verification changes nothing. This is the clearest evidence that a single fabricated source can completely override correct retrieved evidence.

FEVER — verification backfired. FEVER stays robust through Exp 2 (90% accuracy, 0% hallucination) — the model largely ignored the fake "correction notice". But turning on verification in Exp 3 *hurt*: accuracy fell from 90% to 60% and hallucination rose to 33%. The skeptical prompt appears to make the model second-guess correct verdicts, pushing it toward NEI or toward the poison.

HotpotQA — noisy and inconclusive. Accuracy is a flat 50% across all three experiments (multi-hop questions are hard for this setup), while hallucination climbs 30% → 50% → 80%. As §4 explains, the 30% baseline is largely spurious, so these numbers should be read with caution.

4. Concluding remarks

4.1 Discussion of findings

Returning to the two research questions: **RQ1 is answered clearly in the affirmative** — injected misinformation can beat RAG. The synthetic result (0% → 100% hallucination from a single planted document) is unambiguous, and it aligns with the broader knowledge-base-poisoning literature: when retrieval surfaces a fabricated passage, a well-aligned model tends to trust its context and repeat it.

RQ2 is answered in the negative for this run. The verification prompt did not recover the poisoned case on any dataset; it was neutral on synthetic and actively *harmful* on FEVER and HotpotQA, where it lowered accuracy and raised hallucination. The plausible mechanism is over-correction: instructing the model to distrust its sources makes it abandon correct evidence as readily as incorrect evidence, since the prompt gives it no reliable way to tell them apart. A purely prompt-level defense, at least in this lightweight form, is therefore insufficient.

Overall verdict. The experiments confirm that injected misinformation can beat RAG (RQ1), but they do **not** support the hypothesis that a verification prompt defends against it (RQ2) — in this run the prompt was neutral at best and harmful at worst.

4.2 Limitations and threats to validity

Several weaknesses mean the *direction* of the findings is trustworthy but the exact percentages are not:

1. **Substring scoring produces false positives.** Hallucination is measured by checking whether the fake-answer string appears anywhere in the model's text. For HotpotQA the fake answer is just another real answer from the dataset (often a common word), so the model can match it by coincidence. This is why HotpotQA shows 30% hallucination at *baseline* — an experiment that contains **no poison at all**. That 30% is a noise floor, not real hallucination.
2. **An answer can be scored correct *and* hallucinated at once.** Example from synthetic Exp 2:

```
Q: formation year of Quantum Echo? (gold 2011, poison 2008)
A: "...either 2008 (Source 1) or 2011 (Source 2) ... I don't know the correct
answer due to this discrepancy."
→ is_correct = True (contains 2011) AND is_hallucinated = True (contains
2008).
```

The model actually behaved *sensibly* — it flagged the contradiction — yet the metric counts it as a full hallucination. So "100% hallucination" on synthetic overstates the failure.

3. **Tiny sample.** Only 10 items per dataset, so each question is worth 10 percentage points; the swings are within noise.
4. **Single model, single retrieval setting.** One model and one `top_k` were tested; the framework supports sweeping both, but this run did not.

4.3 Future work

- **Fix the metric first.** Replace substring scoring with exact/normalized answer matching or an LLM judge, and separate "flagged the contradiction" from "repeated the lie" as distinct outcomes. This is the prerequisite for trusting any comparison.
- **Scale and sweep.** Move to 50–100 items per dataset and sweep multiple models and `top_k` values for statistically meaningful results, optionally with self-consistency to measure answer stability.

- **Stronger defenses.** Compare the prompt-level verification against retrieval-side defenses — provenance/trust scoring of sources, majority-vote across retrieved passages, or a separate cross-checking model — to see whether any actually recover the poisoned case.
- **Attack strength.** Vary the number and ranking of poison passages to map how robustness degrades as the attacker controls more of the top- k .

4.4 Conclusion

This project demonstrates a concrete data-poisoning attack on RAG: a lone fabricated document can flip a correct system into confidently repeating false information. The proposed prompt-level defense was not effective in this evaluation. The most valuable next step is repairing the scoring methodology, after which the verification hypothesis can be re-tested at larger scale and against stronger, retrieval-side defenses with confidence.

Reproduce: install CPU-only PyTorch then `requirements.txt`; add `GROQ_API_KEY` to `.env`; run `python -m src.evaluator` (writes the result CSVs; first run downloads HotpotQA and FEVER). Figures regenerated from `results/experiment_results.csv` (90 rows, llama-3.3-70b-versatile, top_k=4) via `notebooks/demo.ipynb`.